

k-NN Imputation of Missing Values

22118 Project Report

Authors

Bhavya (s252641)

Magnus (s190867)

Course

22118 - Advanced Unix Python for Bioinformaticians

Date

May 10, 2026

Contents

1	Introduction	3
2	Contribution	3
3	Theory	3
3.1	k-Nearest Neighbor Imputation	3
3.2	Root Mean Square Error (RMSE)	4
4	Algorithm Design	4
4.1	Core k-NN Algorithm	4
4.2	Missing Value Generation	5
5	Program Design	6
5.1	Program Structure	6
5.2	Core Functions	6
5.3	Data Flow	7
6	Program Manual	8
6.1	Installation	8
6.2	Usage	8
6.3	Input Format	9
6.4	Output	9
7	Results	10
7.1	Quantitative Results	10
7.2	Visual Analysis	11
7.3	Key Observations	11
8	Runtime Analysis	12
8.1	Theoretical Complexity	12
8.2	Empirical Performance	13
8.3	Scalability	13
9	Unit Testing	13
9.1	Tests Performed	13
9.2	Results	14
10	Discussion	14
10.1	Optimal Parameter Selection	14
10.2	Effect of Missing Data Percentage	14
10.3	Computational Limitations	14
10.4	Validation Approach	14
10.5	Limitations	14
11	Conclusion	15
11.1	Main Findings	15
11.2	Practical Recommendations	15
11.3	Limitations and Future Work	15
12	References	16

13 Appendix	16
13.1 GitHub	16
13.2 Development Process: Iterations and Commits	16
13.3 AI Use Declaration	16

1 Introduction

Missing values are common in datasets such as microarray experiments, where measurements may fail or be unreliable. Many analysis methods require complete data matrices. This makes it vital to many analyses to be able to estimate missing values. In this project, missing values are imputed using the *k-nearest neighbors (k-NN)* algorithm.

The goal of the project is to evaluate how accurately any missing values can be reconstructed under different conditions, such as the proportion of missing data. The performance of the method is assessed by removing known values from a complete dataset, estimating them via the k-NN algorithm, and then comparing these estimates to the original values in the dataset.

2 Contribution

This project was completed as a collaboration between the two authors on a principally equal level, with equal contributions across all major components, including planning, research, code writing, and report writing. The specific work areas below were not perfectly siloed, and both authors have contributed and edited each other's work.

Magnus (s190867):

- Data preprocessing
- Writing pipeline for k-NN implementation

Bhavya (s252641):

- Writing and running experimental work implementing pipeline
- Writing and running plotting

All design decisions, debugging, report revisions, as well as any other decision, were made jointly and in close collaboration. Any and all coordination was through text messages or physical or virtual meetings.

The report was written jointly and concurrently in Overleaf, the online live LaTeX-editor.

3 Theory

3.1 k-Nearest Neighbor Imputation

The k-NN algorithm estimates missing values by finding similar patterns in the data. It assumes rows with similar values across columns will have similar values at missing positions [3].

Distance Metric: Similarity between two gene expression profiles **a** and **b** is measured using Euclidean distance:

$$d(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_{i \in M} (a_i - b_i)^2} \quad (1)$$

where M includes only positions where both vectors have non-missing values. If no common positions exist, $d = \infty$ and that row is excluded as a neighbor.

Imputation Process: For each missing value at position (i, j) :

1. Find all rows r where column j has a value
2. Calculate $d(\text{row}_i, \text{row}_r)$ for each candidate

3. Select the k rows with smallest distances
 4. Impute as the mean of these k neighbors' values at column j
- If no valid neighbors exist, use the column mean.

3.2 Root Mean Square Error (RMSE)

RMSE quantifies imputation accuracy by measuring prediction errors. For n artificially removed values:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{(i,j) \in S} (x_{ij}^{\text{original}} - x_{ij}^{\text{imputed}})^2} \quad (2)$$

where S is the set of removed positions. RMSE is appropriate because it penalizes large errors (due to squaring), uses the same units as the data, and is standard in imputation validation literature [2, 3].

4 Algorithm Design

4.1 Core k-NN Algorithm

The k-NN imputation algorithm processes each missing value independently by finding similar gene expression patterns. The algorithm is structured in the following way:

Algorithm 1: k-NN Imputation

Listing 1: k-NN Imputation Pseudocode

```

Input: data matrix D (n rows n x m columns), integer k
Output: imputed matrix D'

D' = copy of D

for each row i in D:
    for each column j in D:
        if D[i,j] is missing:
            neighbors = []

            # Find k nearest neighbors
            for each row r in D (where r != i):
                if D[r,j] is not missing:
                    dist = euclidean_distance(D[i], D[r])
                    neighbors.append((dist, D[r,j]))

            # Sort by distance and select k nearest
            neighbors.sort(by distance)
            k_nearest = neighbors[0:k]

            # Impute as mean of neighbor values
            if k_nearest is not empty:
                D'[i,j] = mean([value for (dist, value) in k_nearest])
            else:
                D'[i,j] = column_mean(D[:,j])

return D'

```

Step-by-Step Explanation:

1. **Iterate through matrix:** Scan each position to identify missing values
2. **Compute distances:** For each missing value at (i, j) , calculate the Euclidean distance from row i to all other rows r that have a non-missing value at column j
3. **Select neighbors:** Sort candidate rows by distance and select the k closest
4. **Impute value:** Replace missing value with the arithmetic mean of the k neighbors' values at column j
5. **Fallback strategy:** If no valid neighbors exist (all distances infinite), use the column mean as a conservative estimate

Design Decisions:

- **Euclidean distance:** Simple and effective for capturing value similarity in continuous gene expression data. Is also the standard in the literature [3].
- **Mean aggregation:** Simple arithmetic mean gives good estimates and is not very sensitive to outliers when k is moderate (5-15).
- **Row-wise comparison:** Genes (rows) are compared across samples (columns) to identify similar expression patterns, reflecting biologically meaningful gene co-regulation.

4.2 Missing Value Generation

To validate the imputation algorithm, artificial missing values are introduced into complete (no missing values) sets of data. This allows comparison between imputed and true values.

Algorithm 2: Missing Value Generation

Listing 2: Missing Value Generation Pseudocode

```

Input: complete data matrix D, percentage p ( $0 < p < 1$ )
Output: corrupted matrix D_corrupt, indices of removed values

set random seed to 42  # for reproducibility

D_corrupt = copy of D
total_values = n_rows x n_columns
n_missing = floor(total_values x p)

# Randomly select positions to remove
random_indices = random_sample(total_values, n_missing, without
    replacement)
row_indices, col_indices = convert_to_2D_indices(random_indices)

# Set selected positions to NaN
for each (i, j) in (row_indices, col_indices):
    D_corrupt[i, j] = NaN

return D_corrupt, (row_indices, col_indices)

```

Ensuring Reproducibility: The algorithm uses a fixed random seed (42) to ensure that:

- results are reproducible across multiple runs
- different parameter settings (k values) can be fairly compared on identical missing patterns
- validation results can be verified and replicated

Missing values are uniformly distributed across the matrix, ensuring no systematic bias in which genes or samples are affected.

5 Program Design

5.1 Program Structure

The implementation is organized into four Python scripts that provide modular functionality:

- **preprocess.py**: Loads and formats data from raw microarray files
- **pipeline.py**: Imputes values and performs standalone testing
- **experiment.py**: Does experimental evaluation with a range of parameter values
- **plots.py**: Visualizes results and generates figures

The modular design separates the entire analysis into discrete steps of data preparation, algorithm implementation, systematic evaluation, and visualization. These are independent components that can be tested and modified separately.

5.2 Core Functions

The pipeline script consists of five primary functions that together perform k-NN imputation and validation:

load_data(filename)

- **Purpose**: Loads tab-separated gene expression data
- **Input**: Path to data file (tab-separated format with Probe_ID column)
- **Output**: NumPy array of expression values (excludes Probe_ID column)
- **Function**: Reads matrix data, converts 'NA' strings to np.nan, skips header and ID column

introduce_missing(data, percent)

- **Purpose**: Generates artificial missing values for validation
- **Input**: Complete data matrix, percentage of values to remove (0-1)
- **Output**: Tuple (corrupted matrix, indices of removed values)
- **Function**: Randomly selects positions and replaces with np.nan (using fixed seed (42) for reproducibility)

knn_impute(data, k)

- **Purpose**: Performs k-NN imputation on missing values
- **Input**: Data matrix with missing values (np.nan), number of neighbors k
- **Output**: Imputed matrix with all missing values filled
- **Function**: For each missing value, finds k nearest neighbor rows based on Euclidean distance, imputes as mean of neighbor values

compute_rmse(original, imputed, missing_indices)

- **Purpose**: Calculates prediction error on artificially removed values

- **Input:** Original complete matrix, imputed matrix, indices of removed values
- **Output:** Float RMSE
- **Function:** Compares original and imputed values only in the positions that were artificially removed

`run_experiment(data, missing_percents, k_values)`

- **Purpose:** Systematic evaluation across parameter ranges
- **Input:** Complete data, list of missing percentages, list of k values to test
- **Output:** Dictionary mapping percentages to lists of RMSE values
- **Function:** Generates one corrupted dataset per percentage, tests all k values on the same corrupted data for fair comparison

5.3 Data Flow

The program execution follows this data flow:

Listing 3: Data Flow Pseudocode

```
# Preprocess (one-time setup)
raw_files -> preprocess.py -> data.txt

# Experimental pipeline
data.txt -> load_data() -> complete_matrix

for each missing_percentage:
    complete_matrix -> introduce_missing() -> (corrupted_matrix, indices)

    for each k_value:
        corrupted_matrix -> knn_impute(k) -> imputed_matrix
        (complete_matrix, imputed_matrix, indices) -> compute_rmse() ->
            RMSE

    store RMSE values

all_results -> plot_results() -> knn_results.png
```

Key Design Features:

- **Separation of concerns:** Each function has a single, well-defined responsibility
- **Reproducibility:** Fixed random seed ensures consistent results
- **Fair comparison:** Same corrupted dataset used for all k values within each missing percentage
- **Modularity:** Functions can be imported and reused (e.g., `experiment.py` imports from `pipeline.py`)
- **Command-line interface:** `pipeline.py` supports `argparse` for flexible parameter specification from command line (instead of hard-coding)

The program can be executed in three modes:

1. **Standalone testing:** Run `pipeline.py` with specific parameters
2. **Systematic evaluation:** Run `experiment.py` to test parameter ranges
3. **Complete analysis:** Run `plots.py` to execute experiments and generate visualizations

6 Program Manual

6.1 Installation

Version usage:

- Python 3.13.9
- NumPy 2.3.5
- Matplotlib 3.10.6

Installation steps:

```
# Install required packages
pip install numpy matplotlib

# Verify installation
python -c "import numpy, matplotlib; print('Success')"
```

All scripts should be placed in one directory with the `data.txt` file.

6.2 Usage

The program provides three execution modes with different levels of control:

1. Standalone Pipeline (Single Test)

Run a single imputation test with custom parameters, for example:

```
python pipeline.py -file data.txt -k 10 -missing 0.1
```

Command-line arguments:

- `-file`: Input data file (default: `data.txt`)
- `-k`: k number of nearest neighbors (default: 10)
- `-missing`: Percentage of values to remove, 0-1 (default: 0.1)

Examples:

```
# Test with 5% missing values and k = 15
python pipeline.py -k 15 -missing 0.05

# Use different input file
python pipeline.py -file mydata.txt -k 20 -missing 0.2
```

2. Systematic Experiment

Run experiments across multiple parameter combinations (edit parameters in script):

```
python experiment.py
```

Parameters are defined in the block `if __name__ == "__main__":`. Modify `missing_percentages` and `k_values` lists as needed.

3. Complete Analysis with Visualization

Run full experimental evaluation and generate plots:

```
python plot.py
```

This runs the experiment across all parameter combinations and outputs visualizations. Parameters are configured in the script (default: 5 missing percentages $\times$ 6 k values = 30 test runs).

6.3 Input Format

The input file must be a .tsv file of the structure:

Probe_ID	rep1	rep2	rep3
100002	68.42	89.72	76.86
100003	0.12	0.17	0.09
100027	0.12	0.18	0.13
...			

Format specifications:

- **Delimiter:** Tab character (`\t`)
- **Header row:** First row contains column names
- **First column:** Probe/gene identifiers (strings)
- **Data columns:** Numeric expression values (floats)
- **Missing values:** Represented as 'NA' (converted to `np.nan` internally)
- **Encoding:** UTF-8 text

The `preprocess.py` script converts raw microarray files into this format.

6.4 Output

The program generates multiple outputs depending on execution mode:

Terminal Output:

- Progress indicators (e.g., "Row 50/32878")
- RMSE values for each parameter combination
- Summary tables of results

Example terminal output:

```
Loading data...
Data shape: (32878, 3)

--- Testing 5.0% missing ---
k=5... RMSE=2.285
k=10... RMSE=2.314

--- Testing 10.0% missing ---
k=5... RMSE=2.495
k=10... RMSE=2.521
```

File Outputs:

- **knn_results.png**: Plot showing RMSE vs k for different missing percentages (high-resolution, 300 DPI)
- **results.txt**: Text file containing RMSE values in dictionary format

Plot characteristics:

- X-axis: k (number of neighbors)
- Y-axis: RMSE (root mean square error)
- Multiple lines: One per missing percentage tested
- Grid lines for readability
- Legend indicating missing percentages

Results file format (results.txt):

0.05:	[2.285, 2.298, 2.307, 2.314, 2.319, 2.323]
0.10:	[2.495, 2.508, 2.515, 2.521, 2.525, 2.529]
0.15:	[2.687, 2.701, 2.709, 2.716, 2.721, 2.726]

Each line has a missing percentage followed by RMSE values corresponding to tested k values.

7 Results

The k-NN imputation algorithm was evaluated across five missing data percentages (5%, 10%, 15%, 20%, 25%) and five k values (3, 5, 10, 15, 20). Each configuration was tested on 1500 genes with three replicate (rep1-3). RMSE was calculated by comparing imputed values against the known original values that were artificially removed. Data come from Montserrat et al., 2010 [1].

7.1 Quantitative Results

Table 1 presents the RMSE values for all tested parameter combinations.

Table 1: RMSE values for different missing percentages and k values

Missing %	k=3	k=5	k=10	k=15	k=20
5%	12.25	12.64	12.75	11.98	11.72
10%	11.10	9.84	9.05	8.84	9.75
15%	11.14	10.29	9.14	8.81	8.92
20%	20.86	14.07	10.18	9.50	9.94
25%	20.72	14.44	11.89	11.98	12.08

Optimal configurations:

- **Best overall performance:** 15% missing with $k=15$ (RMSE = 8.81)
- **Best for low missing data (5%):** $k=20$ (RMSE = 11.72)
- **Best for moderate missing data (10-15%):** $k=15$ (RMSE = 8.84 and 8.81)
- **Best for high missing data (20-25%):** $k=15$ (RMSE = 9.50 and 11.98)

7.2 Visual Analysis

Figure 1 illustrates the relationship between k and imputation accuracy across different missing data percentages.

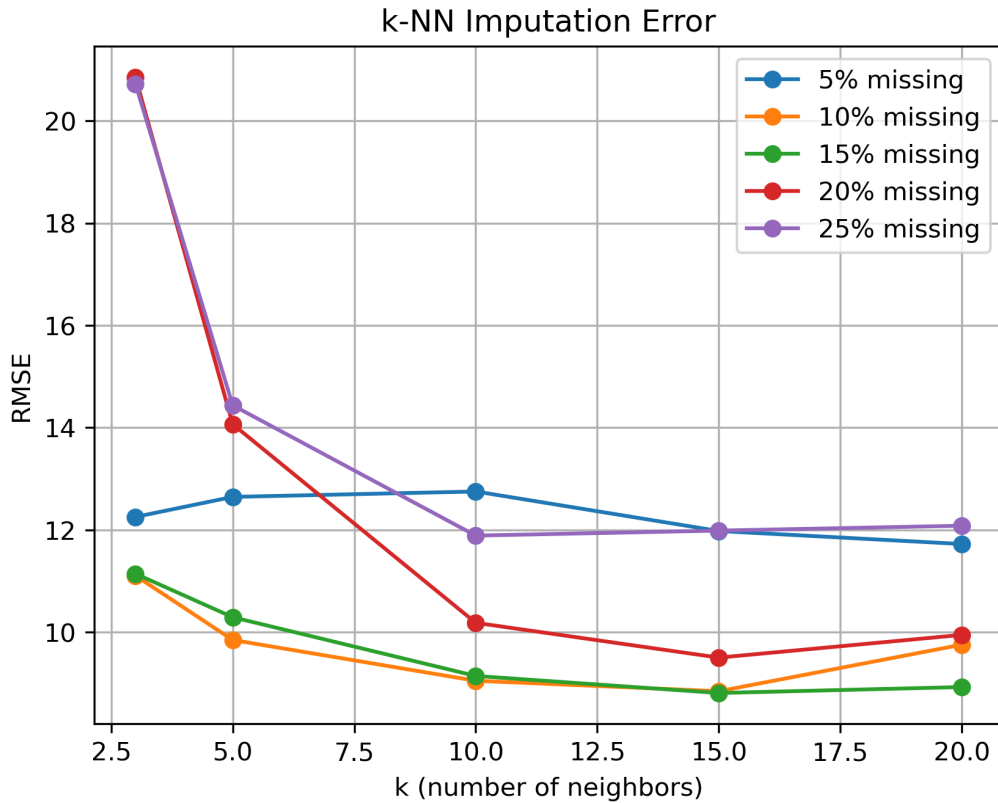


Figure 1: RMSE vs number of neighbors (k) for different percentages of missing data. Each line represents a different missing data percentage, showing how imputation error changes with neighborhood size. Drawn using Matplotlib.

7.3 Key Observations

Effect of k value:

- **Small k ($k=3$):** High RMSE for all missing percentages, particularly severe for 20-25% missing ($\text{RMSE} > 20$). Small neighborhoods are unstable and sensitive to outliers.
- **Moderate k ($k=5-15$):** RMSE generally decreases as k increases, with most configurations showing optimal performance around $k=15$.
- **Large k ($k=20$):** Performance stabilizes or slightly degrades, suggesting over-smoothing begins to reduce accuracy.

Effect of missing percentage:

- **5% missing:** Moderate RMSE (11-13 range) with relatively stable performance across k values. Unexpectedly higher than 10-15% missing.
- **10-15% missing:** Best overall performance (RMSE as low as 8.81), showing k -NN works optimally with moderate missing data.

- **20-25% missing:** Dramatic performance degradation at low k values ($\text{RMSE} > 20$), but substantial improvement with larger neighborhoods. At $k=15-20$, performance approaches that of lower missing percentages.

Convergence behavior:

All missing percentages show convergence toward similar RMSE values (approximately 9-12) as k increases beyond 10. This suggests that with sufficient neighbors, the algorithm can compensate for higher levels of missing data. The most dramatic improvement occurs for high missing percentages (20-25%), where RMSE drops by approximately 50% when moving from $k=3$ to $k=15-20$.

Unexpected pattern:

Counterintuitively, 5% missing data showed higher RMSE than 10-15% missing. This may be due to:

- Random variation in which specific values were removed
- The particular expression patterns in the selected 1500 genes
- Statistical noise from the smaller number of missing values being evaluated

Practical recommendation:

For microarray imputation on similar datasets, $k=15$ provides the most robust performance across all missing data scenarios, achieving RMSE values between 8.81 and 11.98. This neighborhood size balances local pattern sensitivity with stability from sufficient sample averaging.

8 Runtime Analysis

8.1 Theoretical Complexity

The k -NN imputation algorithm shows quadratic time complexity. For a dataset with n rows (genes) and m columns (samples):

Time Complexity:

- Iterate over all positions: $n \times m$
- Compare against all rows for each missing value: $\times n$
- Calculate distance over columns: $\times m$

Only a fraction p will be missing, giving:

$$T(n, m, p) = O(pn^2m^2) \quad (3)$$

Space Complexity: Storage needed:

- Input matrix: $O(nm)$
- Output matrix: $O(nm)$
- Distance list (temporary): $O(n)$

The $O(n)$ term disappears because nm dominates when $m > 1$, giving a total rate of:

$$O(nm) \quad (4)$$

8.2 Empirical Performance

The implementation was tested on 1500 genes across 3 samples with 25 parameter combinations (5 missing percentages $\times$ 5 k values).

Measured performance:

- Total runtime: 101.9 minutes (1.7 hours)
- Average per configuration: 244.4 seconds (4.1 minutes)
- Processing rate: approximately 6 genes/second

8.3 Scalability

The quadratic scaling limits practical application to large datasets:

- Limited run: $n = 1500$ rows in 102 minutes
- Full dataset: $n = 32878$ rows
- Scaling factor: $(32878/1500)^2 \approx 480$
- Estimated full runtime: $102 \times 480 \approx 816$ hours (34 days)

Other methods would be required to process complete microarray datasets, e.g., optimizations such as k-d trees, parallel processing, or approximate methods would be necessary. However, the current implementation of the K-NN algorithm is practical for smaller datasets and to show proof-of-concept, as in this educational context.

9 Unit Testing

A test suite (`tests.py`) was implemented to verify the correctness of core functions through automated unit tests. All tests passed successfully, confirming implementation correctness.

9.1 Tests Performed

Distance function (3 tests):

- Identical vectors return distance 0
- Known Pythagorean triple (3-4-5) correctly computed as 5.0
- No common non-NaN positions returns infinity (critical edge case)

Missing value generation (3 tests):

- Correct percentage of values removed (10% of 300 = 30 values)
- Original data matrix remains unchanged (copy operation verified)
- Indices properly recorded and valid

RMSE calculation (2 tests):

- Zero error case returns $\text{RMSE} = 0$
- Known error values (3-4 triangle) correctly computed as 3.536

9.2 Results

All 8 unit tests passed without errors. No bugs were discovered during testing, confirming that individual functions behave correctly in isolation. The test suite validates both typical use cases and edge cases (NaN handling, data integrity).

10 Discussion

10.1 Optimal Parameter Selection

The experimental results demonstrate that **k=15 provides optimal performance** across most missing data scenarios, achieving the lowest RMSE values (8.81-11.98) for missing percentages from 10-25%. Small k values (k=3) produce unstable results with RMSE exceeding 20 for higher missing percentages, while large k values (k=20) show diminishing returns or slight performance degradation due to over-smoothing.

This aligns with findings in Troyanskaya et al. (2001), where moderate neighborhood sizes balance local pattern sensitivity with statistical stability.

10.2 Effect of Missing Data Percentage

Performance degradation with increasing missing data was most severe at low k values. At k=3, RMSE increased from 12.25 (5% missing) to 20.72 (25% missing). However, with adequate neighborhood size (k=15), the algorithm maintained relatively stable performance even at 25% missing data (RMSE=11.98), demonstrating robustness when properly parameterized.

Unexpectedly, 5% missing data showed higher RMSE than 10-15% across most k values. This counterintuitive result likely reflects random variation in which specific genes had missing values, or the particular expression patterns in our limited 1500-gene subset.

10.3 Computational Limitations

The $O(n^2m^2)$ complexity restricts practical application to moderate-sized datasets. Processing 1500 genes required 102 minutes; extrapolation suggests the full 32,878-gene dataset would require approximately 34 days. This computational burden is the primary trade-off for k-NN's accuracy advantage over simpler methods like row means, as noted by Kim et al. (2004).

For production use, parallelization or approximate nearest neighbor methods would be necessary. However, for one-time analysis of moderate datasets where accuracy is prioritized, the current implementation remains practical.

10.4 Validation Approach

The validation strategy of artificially removing known values and measuring RMSE is appropriate and widely used in the literature. By using the same corrupted dataset across different k values within each missing percentage, fair comparison was ensured. The reproducible random seed (42) enables result verification and replication.

10.5 Limitations

- **Limited dataset:** Testing on 1500 genes may not fully represent performance on the complete 32,878-gene dataset
- **Single distance metric:** Only Euclidean distance was evaluated; Pearson correlation might perform differently

- **Temporal data:** The three time points (0h, 6h, 3 weeks) introduce temporal dependencies not explicitly modeled
- **Sparse comparison:** With only 3 columns, distance calculations have limited information, potentially reducing neighbor quality

Despite these limitations, the implementation successfully demonstrates k-NN imputation as a viable method for microarray missing value estimation, with clear practical guidelines for parameter selection.

11 Conclusion

This project successfully implemented and validated a k-nearest neighbor algorithm to impute missing values in a microarray gene expression data. The project shows that k-NN can handle missing data effectively. Across a range of missing data percentages (5-25%), k-NN performs best at $k = 15$ neighbors.

11.1 Main Findings

- Testing 1500 genes with three time-point samples gave RMSE values from 8.81 to 12.75 for $k = 10$.
- The algorithm proved robust even with substantial missing data (20-25%), achieving acceptable error rates when using adequate neighborhood sizes.
- Troyanskaya et al. (2001) also found that k-NN performs better than simpler methods (e.g., row mean imputation).

11.2 Practical Recommendations

For similar microarray datasets with time-point or replicate structure:

- Use $k = 15$ as a robust default parameter
- Avoid $k \leq 5$ to prevent unstable estimates
- Expect reliable imputation up to 20% missing data
- Account for $O(n^2m^2)$ computational cost in large datasets

11.3 Limitations and Future Work

The primary limitation of this work is scalability. Running the scripts with the full 32,878-gene dataset would take around 34 days with the current setup. Future improvements could include:

- Parallel processing to reduce runtime
- Approximate nearest neighbor methods (k-d trees)
- Alternative distance metrics (Pearson correlation)

The implementation shows k-NN works successfully for microarray imputation on datasets under 1500 genes. In this setup, k-NN works best with the parameter $k = 15$ across most conditions.

12 References

- [1] Montserrat Fitó, Roger Anglada, Rafael de la Torre, Ana Dopazo, Olha Khymenets, Valentini Konstantinidou, Maria-Isabel Covas, Pedro de Alarcón, and Magi Farré-Albaladejo. Human in vivo nutrigenomic effect of olive oil. E-GEOD-10590, 2010. Retrieved 2026-05-10.
- [2] Ki-Yeol Kim, Byoung-Jin Kim, and Gwan-Su Yi. Reuse of imputed data in microarray analysis increases imputation efficiency. *BMC Bioinformatics*, 5(1):160, October 2004.
- [3] Olga Troyanskaya, Michael Cantor, Gavin Sherlock, Patrick Brown, Trevor Hastie, Robert Tibshirani, David Botstein, and Russ B. Altman. Missing value estimation methods for dna microarrays. *Bioinformatics*, 17(6):520–525, June 2001.

13 Appendix

13.1 GitHub

The GitHub repo for this project is public and can be accessed on: https://github.com/s190867/22118_project.

The GitHub repo contains the finished versions of each script as well as other relevant files, such as the original data files, the processed data files, as well as various result files and figures.

13.2 Development Process: Iterations and Commits

Development went forward in rounds through a number of local versions of the scripts pre-processing, pipeline, experiment, and plots. Changes, debugging, and parameter testing were done locally and incrementally before saving stable and polished script iterations into version-controlled milestones (e.g., *pipeline_v1.py*) throughout the project life cycle. These were later pushed to repo in a small number of cycles.

13.3 AI Use Declaration

- Generative AI tools were used during development and report writing, including both Claude and ChatGPT.
- AI was mainly used for feedback, coding support, reformulation, and discussion of implementation approaches.
- AI was also used for trimming and optimizing code before each version commit.
- For writing, AI was used to suggest meaningful and canonical ways of organizing, formatting, and selecting content for the project report.
- All final design decisions, code, analysis, and written content were reviewed and validated by the authors.
- The authors remain fully responsible for the submitted work.